

Projection

Computer Graphics
Instructor: Sungkil Lee

Today

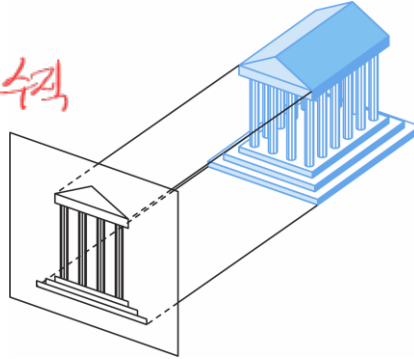
- **Orthographic projection**

- A special case of the parallel projection, where projectors are orthogonal to projection surface.

projector가 projection surface와 수직



멀리 있는 게 작아짐 (diminution)



- **Perspective projection**

- Simple perspective projection
- Symmetric perspective projection in OpenGL
- General non-symmetric perspective projection in OpenGL

Coordinate Systems Revisited

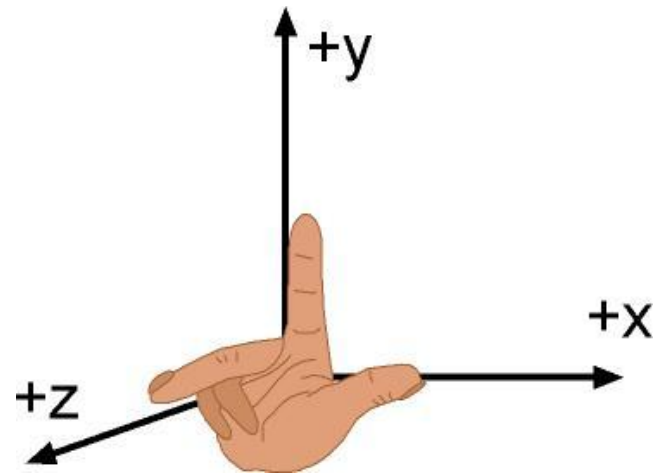
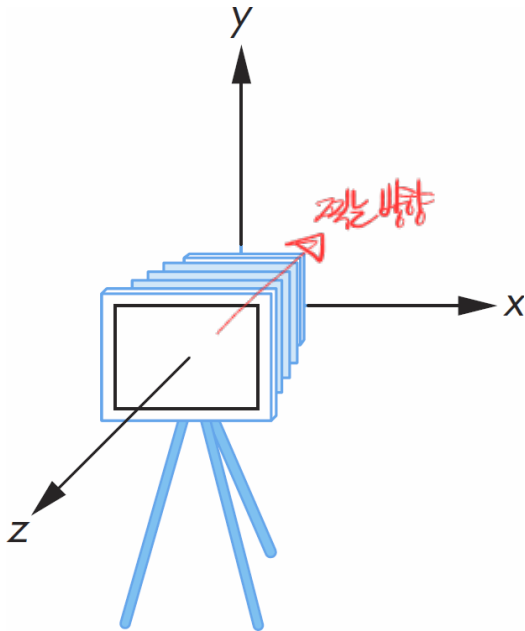
Camera Convention in OpenGL

- **In camera space,**

- a camera is located at the origin, directing in the negative z-direction.
- camera coordinate systems (frames) use RHS convention.
- Recall the following figures.

z가 camera를 향해있음 (오른손)

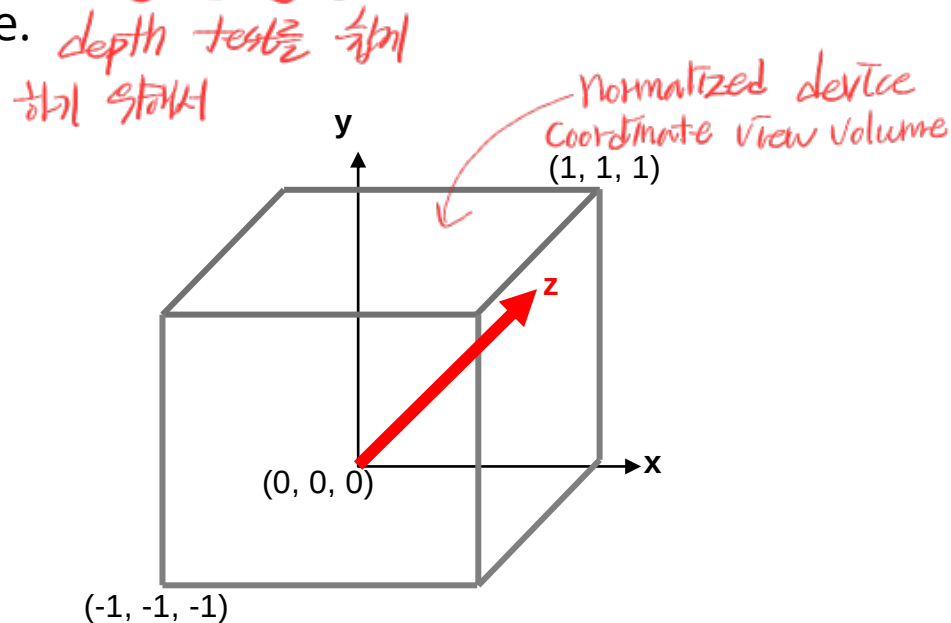
camera는 오른손



Normalized Device Coordinates in OpenGL

- **Normalized Device Coordinate (NDC) System**

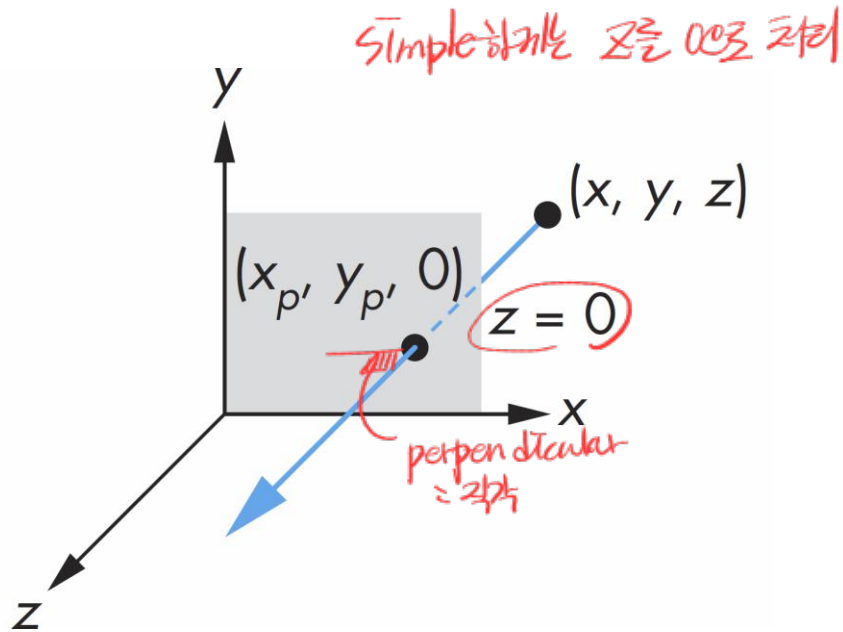
- Through projections for OpenGL, the camera-space points are transformed to NDC.
- Recall that: NDC uses LHS convention: z-axis goes far from your eye
- This is intended for **depth test**, which maintains objects with the smallest depths as visible.



Orthographic Projection Matrix

Orthographic Projection

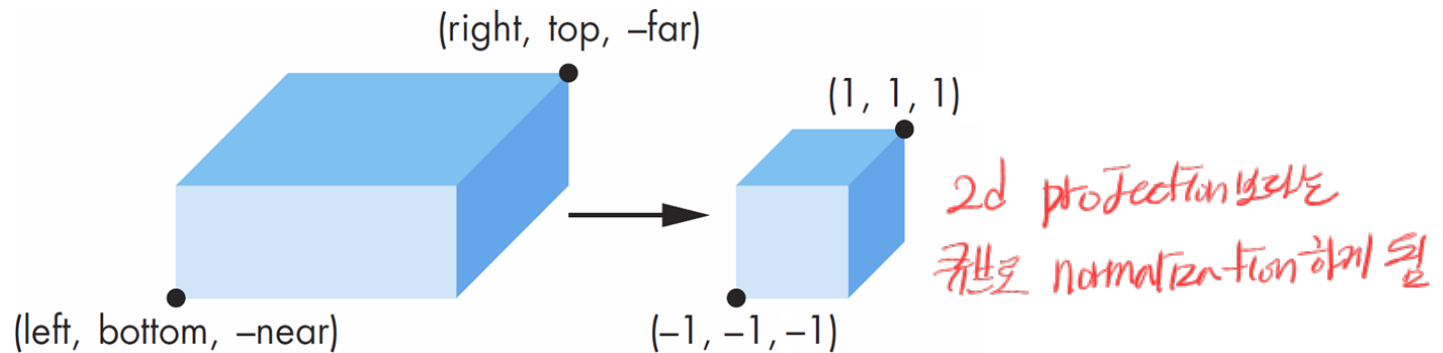
- **A special case of the parallel projection,**
 - in which the projectors are perpendicular to the projection plane
 - corresponds to a camera which has an infinite focal length.
 - Orthogonal projection with the projection plane $z=0$
 - x and y are retained, but only z_p is set to 0.



$$\begin{bmatrix} x_p \\ y_p \\ z_p \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Orthographic Projection in OpenGL

Simple orthographic projection + View volume normalization



• Orthographic projection in OpenGL

- View volume normalization (VVN): all the 3D points are normalized into canonical view volume $[-1,1]^3$ (). *depth test를 위해 z를 0으로*
- **We never explicitly set $z = 0$,** and retain depth information as long as possible in the OpenGL pipeline for **depth test/buffering**.
- That means, even after projection, we still stay in 4D homogeneous coordinates.

View Volume Normalization?

- **Why we do not discard depth after projection?**

- We still need depths to perform depth test (before/after fragment shading) in the pipeline.
- For this, the depth is typically normalized in a fixed range (e.g., [0,1])

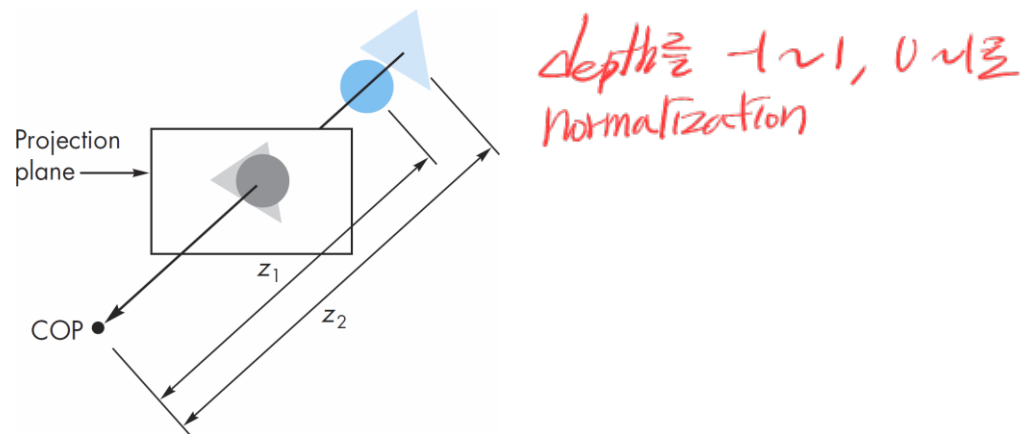


FIGURE 4.41 The z-buffer algorithm.

- **Another benefit of VVN: *efficient clipping***

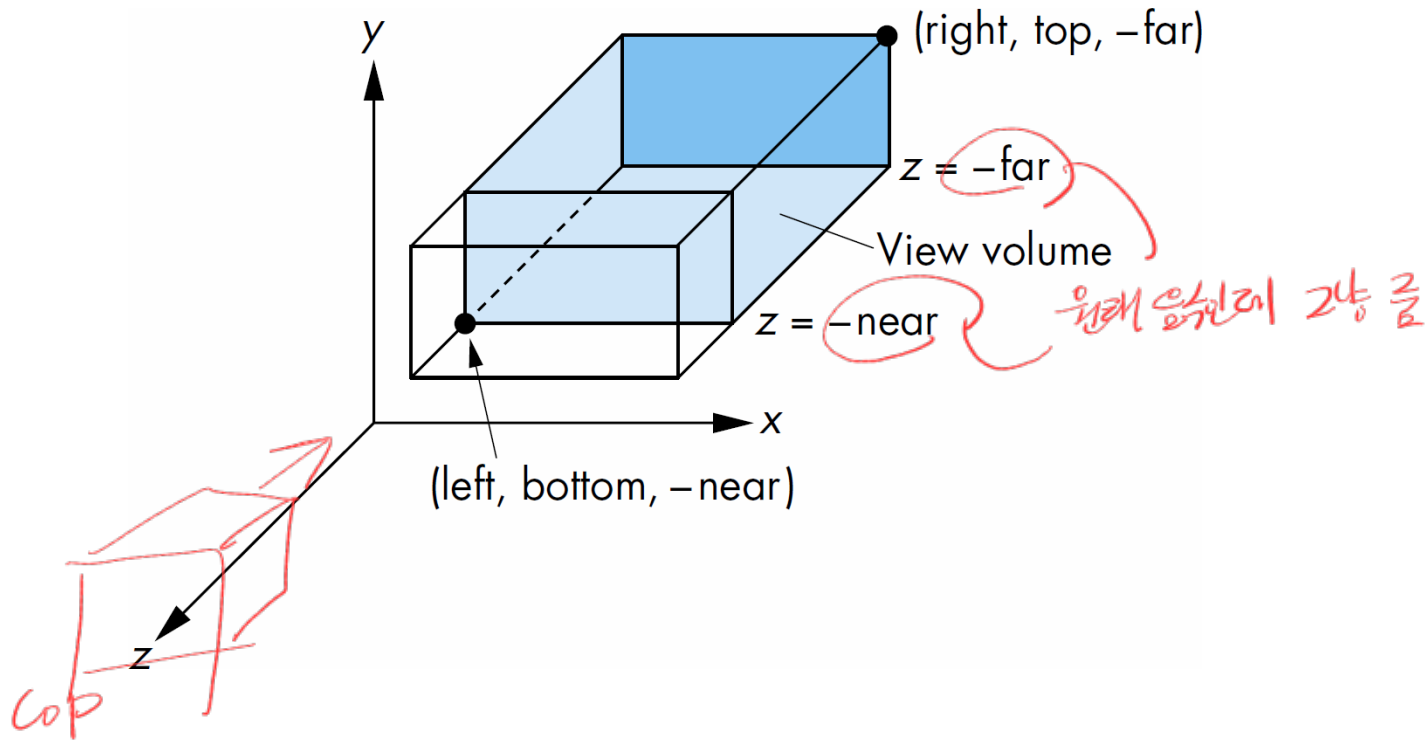
- VVN lets us clip against simple cube regardless of type of projection, including both perspective and orthogonal viewing.

Orthographic Projection in OpenGL

- **We need to implement a function:**

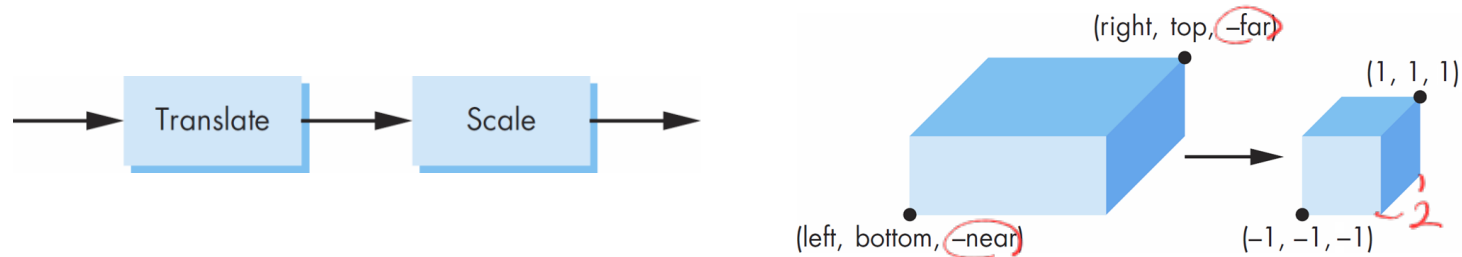
- near and far are depths of clipping planes (i.e., depth bounds)
- $0 < \text{near} < \text{far}$: both are the **positive** distances from camera (COP).

```
mat4 Ortho(left, right, bottom, top, near, far)
```



Orthographic Projection Matrix

- Two-step view volume normalization



- (1) Translation: move center to the origin

$$\mathbf{T} = \mathbf{T}\left(-\frac{\text{left} + \text{right}}{2}, -\frac{\text{bottom} + \text{top}}{2}, +\frac{\text{near} + \text{far}}{2}\right)$$

Handwritten notes: 2D 좌표 (2D coordinates) and 좌표 이동 (coordinate movement)

- (2) Scale to have sides of length 2

$$\mathbf{S} = \mathbf{S}\left(\frac{2}{\text{right} - \text{left}}, \frac{2}{\text{top} - \text{bottom}}, \frac{2}{\text{near} - \text{far}}\right)$$

Orthographic Projection Matrix

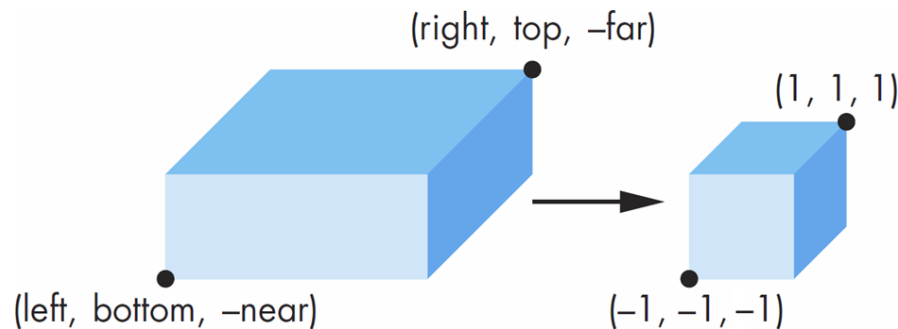
- The combined form of view volume normalization matrix:**

- $(l, r, t, b, n, f) := (\text{left}, \text{right}, \text{top}, \text{bottom}, \text{near}, \text{far})$

$$P = ST = \begin{bmatrix} \frac{2}{r-l} & 0 & 0 & \frac{l+r}{l-r} \\ 0 & \frac{2}{t-b} & 0 & \frac{b+t}{b-t} \\ 0 & 0 & \frac{2}{n-f} & \frac{n+f}{n-f} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Handwritten notes and annotations:

- Scale* (circled around the first three diagonal elements)
- $ST = \frac{2}{r-l} \cdot (-\frac{l+r}{2}) = -\frac{l+r}{l-r}$ (red, pointing to the top-right element)
- $\frac{l+r}{l-r}$ (boxed in red)
- $\frac{b+t}{b-t}$ (boxed in red)
- $\frac{n+f}{n-f}$ (boxed in blue)
- $\frac{2}{n-f} \cdot (\frac{n+f}{2}) = \frac{n+f}{n-f}$ (blue, pointing to the bottom-right element)
- Arrows and text: l, r, b, t 가 상하좌우 방향 (l, r, b, t are up/down, left/right directions)
- Arrows and text: n, f 가 앞뒤 방향 (n, f are front/back directions)



Simple Perspective Projection Matrix

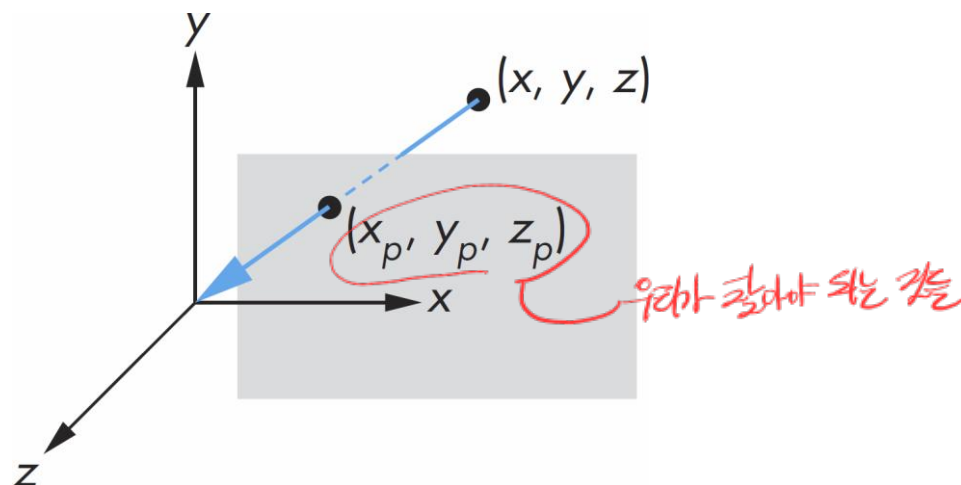
Simple Perspective Projection

- **Our approach:**

- Again, we first consider the mathematics for a simple projection and extend it to the projection for OpenGL, including **view volume normalization (VVN)**.

- **Here, first catch the idea of simple perspective projection.**

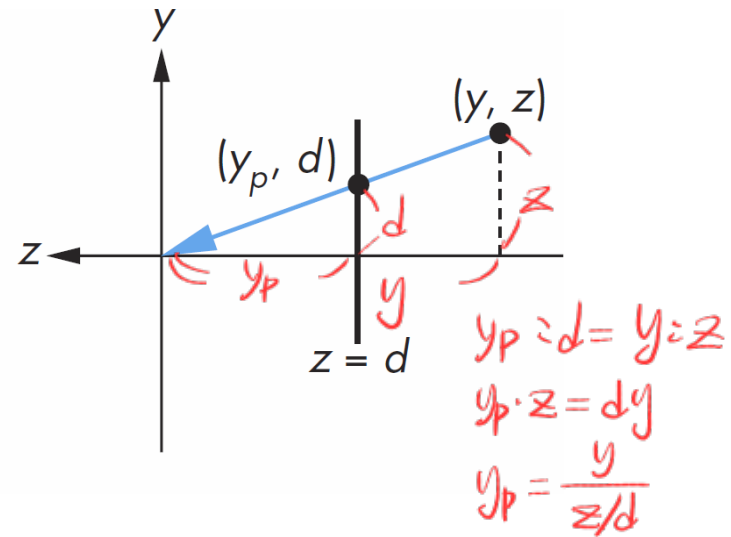
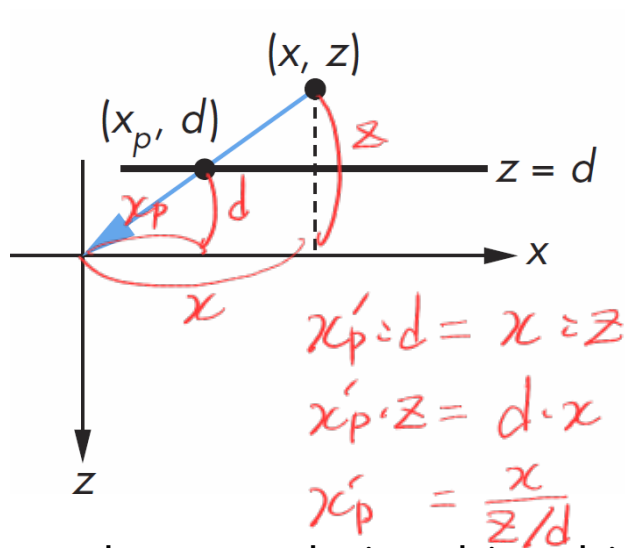
- Set the projection plane in front of the COP.
- The point (x, y, z) is projected into the point (x_p, y_p, z_p) .
- All the projectors pass through the origin (COP).



Simple Perspective Projection

- Set projection plane as orthogonal to z-axis at:

$$z_p = d, \text{ where } d < 0$$



- Then, we have a relationship which describes nonlinear foreshortening:

$$x_p = \frac{x}{z/d}$$

$$y_p = \frac{y}{z/d}$$

Simple Perspective Projection

- The homogeneous coordinate form:

$$\mathbf{M} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1/d & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ z \\ z/d \end{bmatrix}$$

• which makes $\mathbf{p} = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$ to $\mathbf{q} = \begin{bmatrix} x \\ y \\ z \\ z/d \end{bmatrix}$



Simple Perspective Projection

- **Perspective division:**

$$\mathbf{q} = \begin{bmatrix} x \\ y \\ z \\ \textcolor{red}{z/d} \end{bmatrix}$$

- Since $q_w \neq 1$, we must divide by w to return to the Cartesian coordinates. This *perspective division* yields the desired result:

$$x_p = \frac{x}{\textcolor{red}{z/d}},$$

$$y_p = \frac{y}{\textcolor{red}{z/d}},$$

$$z_p = \frac{z}{\textcolor{red}{z/d}} = \textcolor{red}{d}$$

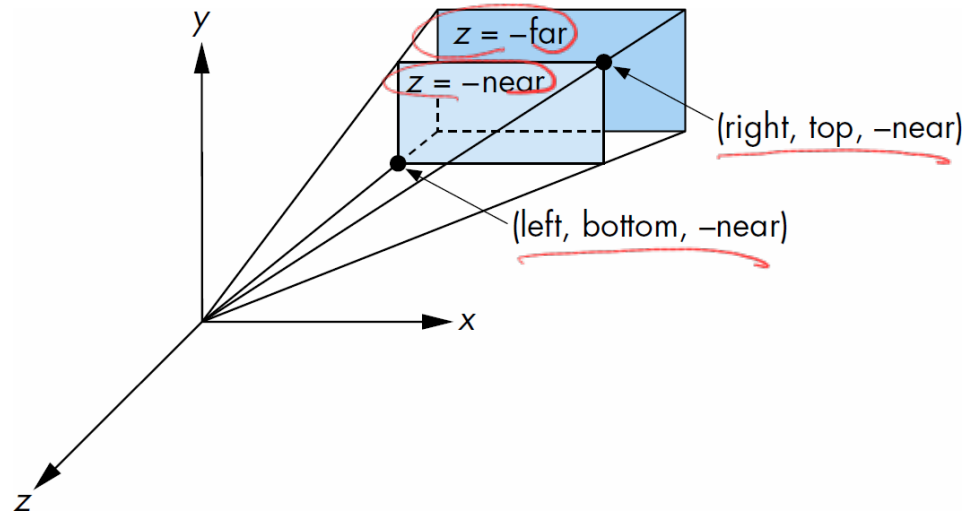
깊이 z에 따라 작아짐

Symmetric Perspective Projection **in OpenGL**

Perspective Projection in OpenGL

Simple perspective projection + **View volume normalization**

- Perspective projection in OpenGL again includes **view volume normalization**.
- For simplicity, we only consider a **symmetric** view volume ($r = -l$, $b = -t$) for the same near and far clipping planes.



Perspective Projection Matrix

frustum을 사육이 depth로 나누기
이렇게 방정식 (depth의 특성과 비례) 때문이 다

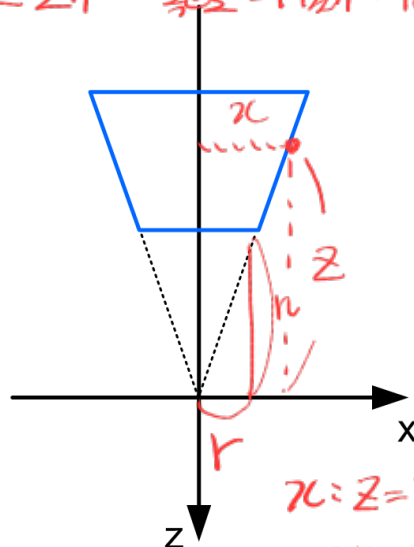
(1) the first step is scaling the frustum to $x = \pm z, y = \pm z,$

- The symmetric input frustum is described as:

$$x = \pm \frac{r}{n} z, \quad y = \pm \frac{t}{n} z, \quad \overset{\text{far}}{-f} \leq z \leq \overset{\text{near}}{-n}$$

scaling 해야 함

z가 -f로 되어있기 때문



$\frac{1}{\frac{r}{n}}, \frac{1}{\frac{t}{n}}$
: scaling factor

↓
1/n, 1/t

$$x = z = t = n$$

$$x \cdot n = z \cdot t$$

$$x = \frac{t}{n} z$$

$$= \pm \frac{t}{n} z \rightarrow$$

$\frac{t}{n}$ = 가늘기
z가 바빠서 z가 같

Perspective Projection Matrix

(1) the first step is scaling the frustum to $x = \mp z, y = \mp z$,

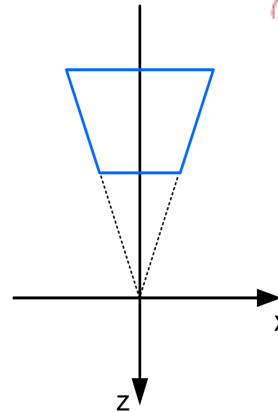
- given the symmetric input frustum

$$x = \mp \frac{r}{n} z, \quad y = \mp \frac{t}{n} z, \quad -f \leq z \leq -n$$

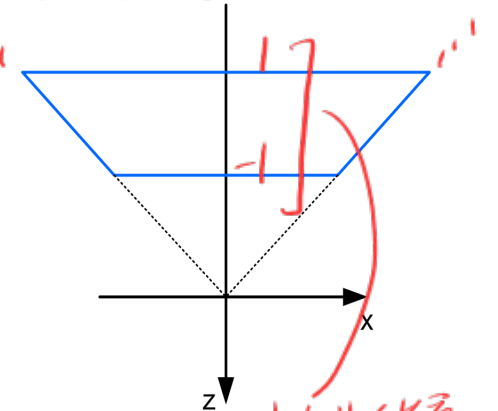
- the scaling matrix can be:

scaling factor

$$S = \begin{bmatrix} n/r & 0 & 0 & 0 \\ 0 & n/t & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



view frustum을 가늘게
1, -1인 형태로 바꿀 것
(x, y만 변경)

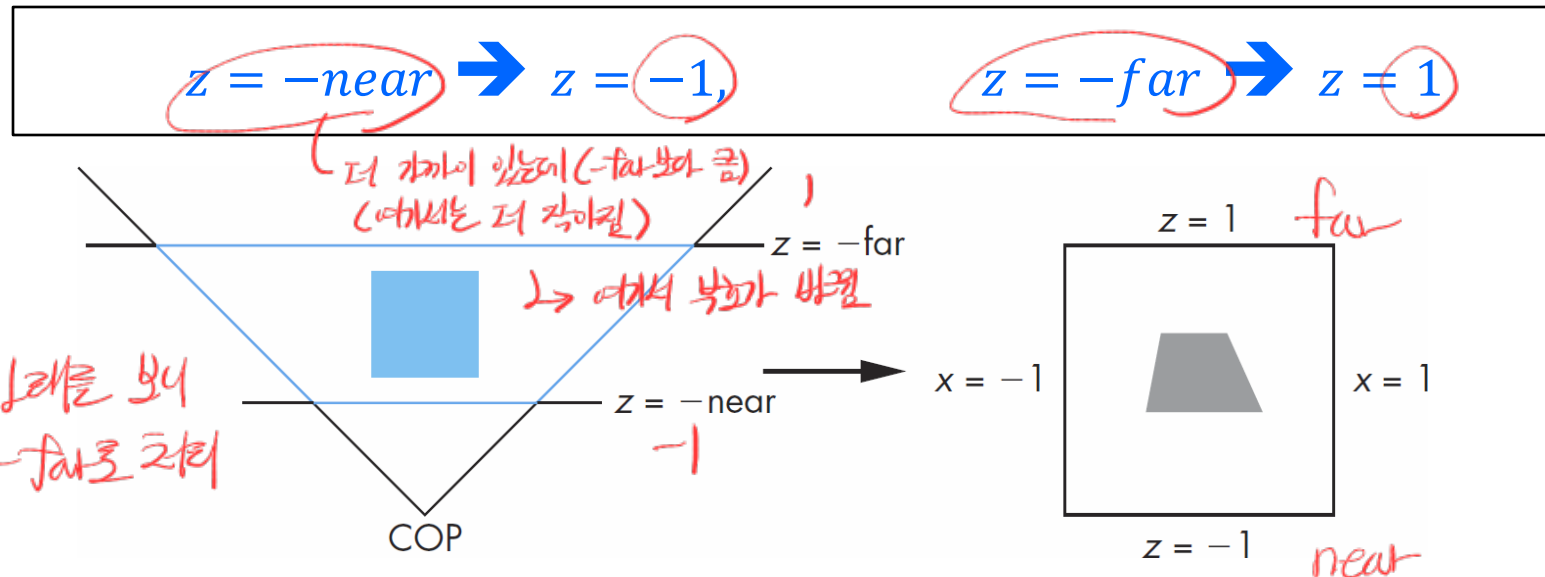


이렇게 좌표
바꿔 줘야 함
→ depth normalization

Perspective Depth Normalization

(2) secondly, perform the perspective projection with depth normalization for the view volume:

- determined by the planes $x = \mp z$, $y = \mp z$ (scaled in the 1st step), and



- Note that the signs of depths are inverted during this conversion, which is intended to maintain the ordering of depths from the viewer.

camera와 normalized device coordinate에서
z가 뒤집어지게 바로 이 단계

normalized한 다음이 z가 -1보다 작거나
1보다 큰 clipping plane 밖으로 안 그림

Perspective Depth Normalization

$\alpha, \beta \Rightarrow$ depth를 반으로
projection할 수 있도록

- Consider a matrix form,

- modified from a simple perspective projection (was $\alpha = 1, \beta = 0$),

여기만 잘들 수 있음

기, y, z 좌표, 원본 좌표

$$N = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \alpha & \beta \\ 0 & 0 & -1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ z\alpha + \beta \\ -z \end{bmatrix}$$

z를 -1로 둘

with 1이 아닐
→ 나눠주기

- After perspective division, the point $(x, y, z, 1)$ goes to

$$(x', y', z') = \left(-\frac{x}{z}, -\frac{y}{z}, -\left(\alpha + \frac{\beta}{z}\right) \right)$$

- For xy clipping planes ($x = \pm z, y = \pm z$),

- $(x', y') \in [-1, 1]^2$ is now **normalized in the canonical view volume** (CVV).
- What remains is normalizing depth clipping planes to CVV.

depth, $[-1, 0]$

Perspective Depth Normalization

- **For near and far clipping planes:**

- $z = -near$, and $z = -far$ are transformed to:

$$z' = -\left(\alpha - \frac{\beta}{near}\right) \quad \text{and} \quad z' = -\left(\alpha - \frac{\beta}{far}\right)$$

- Picking α and β such that $z' = -1$, and $z' = 1$, and solving the equations

$$\textcircled{-1} = -\left(\alpha - \frac{\beta}{near}\right) \quad \text{and} \quad \textcircled{1} = -\left(\alpha - \frac{\beta}{far}\right)$$

- gives us:

$$\boxed{\alpha = \frac{near+far}{near-far}}, \quad \boxed{\beta = \frac{2 \cdot near \cdot far}{near-far}}$$

↙ near, far는 상수
near, far는 constant

near, far는 -1, 1로 맵핑됨
depth normalization 끝

Perspective Depth Normalization

- **Final form:**

- Then, the final form of symmetric perspective projection matrix (including both scaling and normalization) is:

$$\mathbf{P} = \mathbf{N}\mathbf{S} = \begin{bmatrix} n/r & 0 & 0 & 0 \\ 0 & n/t & 0 & 0 \\ 0 & 0 & \frac{n+f}{n-f} & \frac{2nf}{n-f} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

이걸 바꿔줘야 함
(쓰기 전에)

쓰기 이전은 아님
: near, far는 관측점 (작은값으로 인식 가능)
 r, t 는 위치 조정

↓
perspective

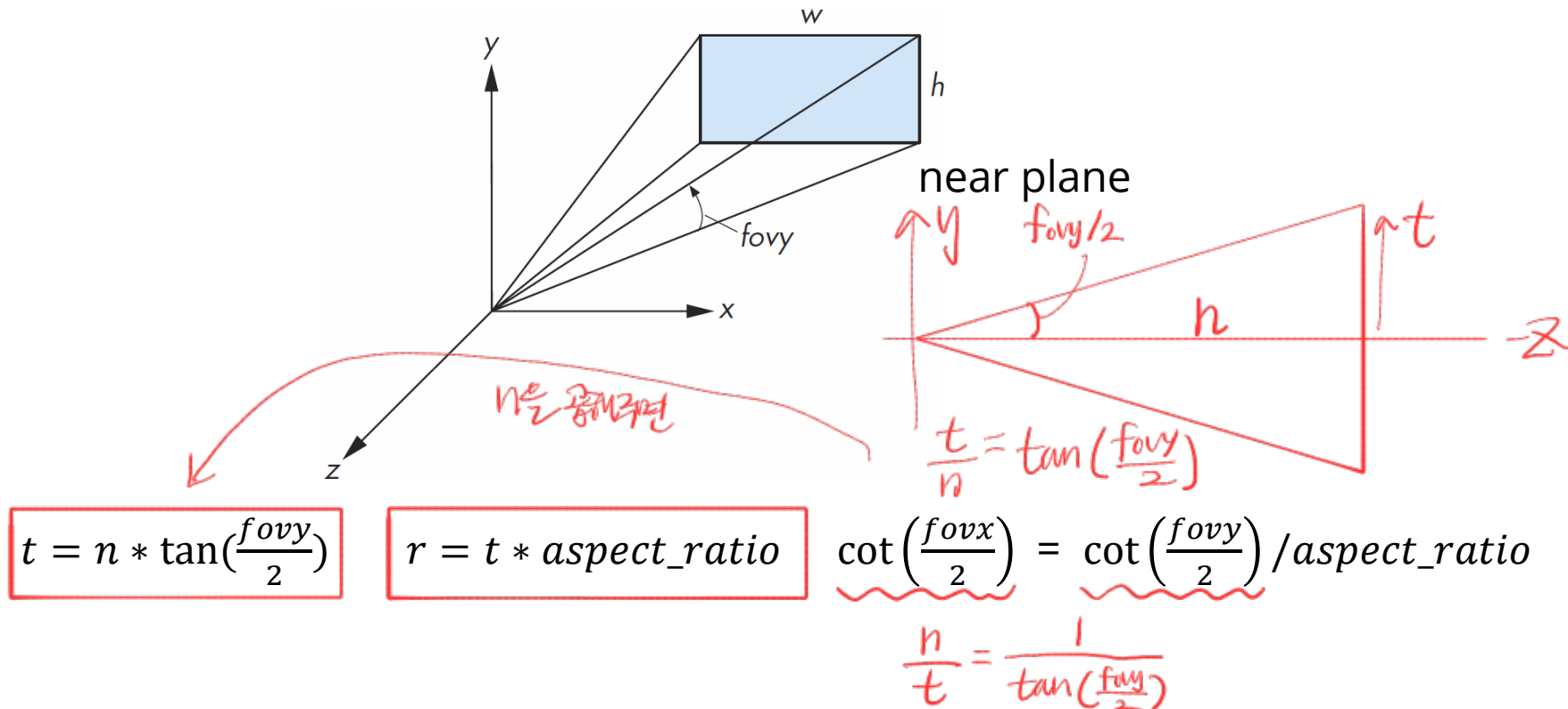
Alternative: Perspective()

l, r, t, b를 주지 않아요 많은 function

- **Having l, r, t, b is often difficult due to their measurement.**

- perspective() method provides more intuitive/controllable projection.
- **fovy** (field of view) and **aspect_ratio** (width/height of sensor):
 - 'y' in fovy: vertical

```
mat4 perspective(fovy, aspect_ratio, near, far)
```



Alternative: Perspective()

l, r, b, t, n, f 를 가 2차원 평면에 대한 여의 parameter인 주면 될

`mat4 perspective(fovy, aspect_ratio, near, far)`

↳ $\frac{h}{w}$

- Then, we have an alternative representation.
- This shape is more common than the frustum (i.e., LRBTNF).

$$P = \begin{bmatrix} \cot\left(\frac{fovx}{2}\right) & 0 & 0 & 0 \\ 0 & \cot\left(\frac{fovy}{2}\right) & 0 & 0 \\ 0 & 0 & \frac{n+f}{n-f} & \frac{2nf}{n-f} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

perspective matrix = $\begin{bmatrix} \cot\left(\frac{fovx}{2}\right) & & & \\ & \cot\left(\frac{fovy}{2}\right) & & \\ & & \frac{n+f}{n-f} & \frac{2nf}{n-f} \\ & & -1 & 0 \end{bmatrix}$ perspective matrix

Appendix: General (Non-Symmetric) Perspective Projection in OpenGL

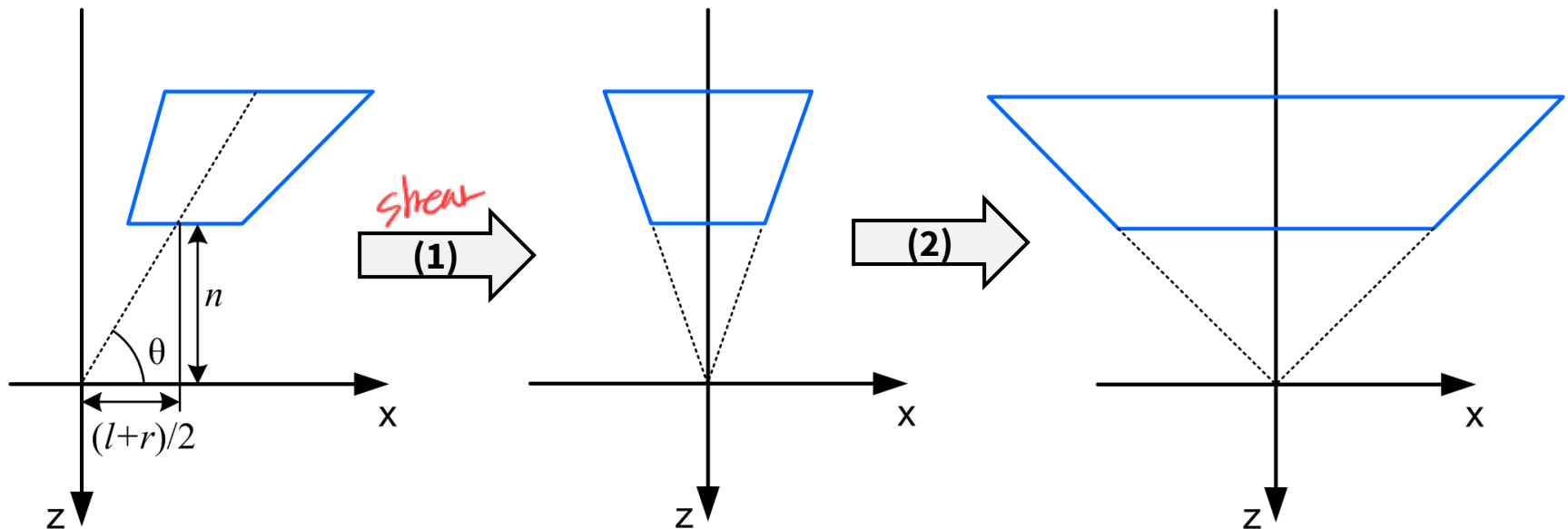
↳ VR 그림 camera가 하나보다 많을 때
(양쪽 눈)

General Perspective Projection in OpenGL

- **Three steps:**

이것은 추가될

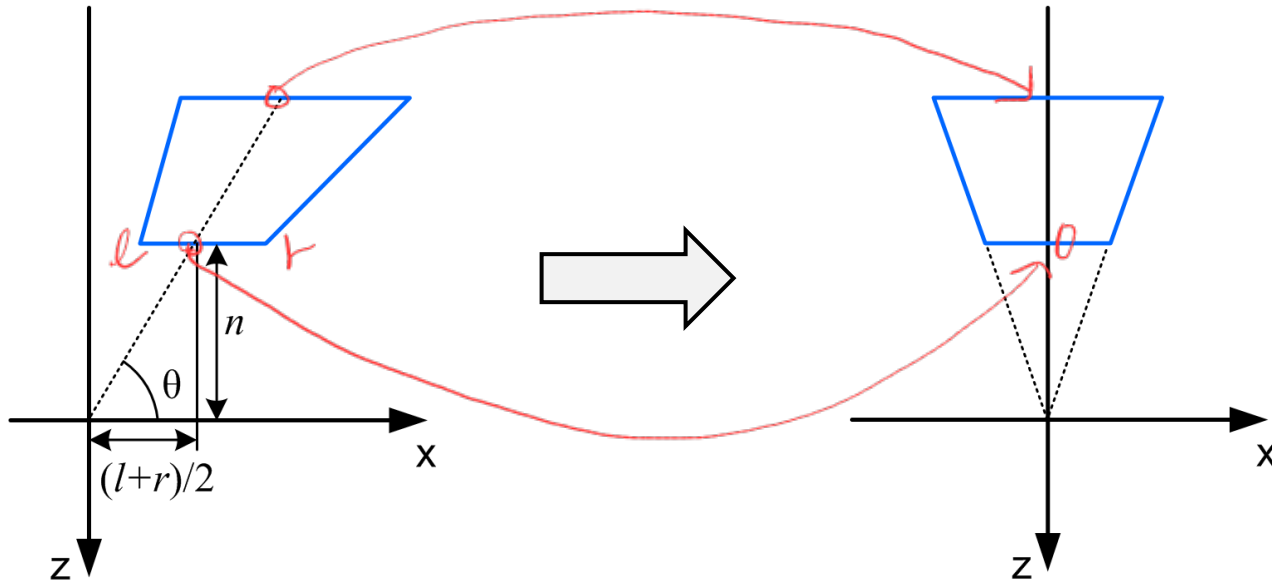
- (1) shear transform to make the symmetric volume (new, here!) ←
- (2) scale the sides of the frustum to $x = \pm z$, $y = \pm z$
- (3) simple perspective projection with *perspective depth normalization*.



General Perspective Projection Matrix

(1) shear transform to make the symmetric volume

- See the top view for x shear (also, do the same for y shear)



General Perspective Projection Matrix

(1) shear transform to make the symmetric volume

- For the shear transform, we skew the points

$$\left(\frac{l+r}{2}, \frac{t+b}{2}, -n \right) \text{ to } (0,0,-n) \quad \text{and} \quad \left(\frac{(l+r)f}{2n}, \frac{(t+b)f}{2n}, -f \right) \text{ to } (0,0,-f)$$

- General cases ($z < 0$) can be written as :

$$\left(-\frac{(l+r)z}{2n}, -\frac{(t+b)z}{2n}, z \right) \text{ to } (0,0,z)$$

- Then, the shear transform matrix (note the inverted signs):

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & (l+r)/2n & 0 \\ 0 & 1 & (t+b)/2n & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

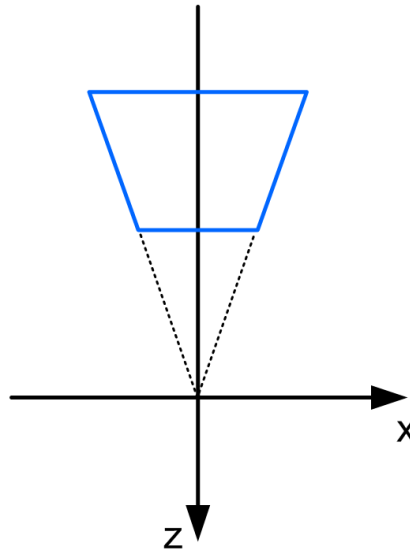
조각 공해량 (z에 바레)

General Perspective Projection Matrix

(1) shear transform to make the symmetric volume

- The resulting frustum is:

$$x = \mp \frac{(r - l)}{2n} z, y = \mp \frac{(t - b)}{2n} z, -f \leq z \leq -n$$



Exercises

Analysis of Perspective Projection Matrix

- Given the symmetric perspective projection matrix,

$$P = \begin{bmatrix} \cot\left(\frac{fovx}{2}\right) & 0 & 0 & 0 \\ 0 & \cot\left(\frac{fovy}{2}\right) & 0 & 0 \\ 0 & 0 & \frac{n+f}{n-f} & \frac{2nf}{n-f} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

$\overset{P_{33}}{\circlearrowleft} \frac{n+f}{n-f} \quad \overset{P_{34}}{\circlearrowleft} \frac{2nf}{n-f}$

- Find the following parameters

- fovy
- aspect ratio
- near and far

$$fovy = 2 \cdot \cot^{-1} \left(\cot \frac{fovx}{2} \right)$$

$$aspect = \frac{\cot\left(\frac{fovx}{2}\right)}{\cot\left(\frac{fovy}{2}\right)}$$

$$near = \frac{P_{34}}{P_{33} - 1}, \quad far = \frac{P_{34}}{P_{33} + 1}$$